

MLSYS Course Project - Phase 1

题目描述、要求拆解与实现指导

供后续交给 Codex 编码实现使用

文档定位	把 Phase 1 的官方要求与老师额外补充要求整理成一份可以直接驱动编码的说明文档。
内容边界	前半部分尽量保持任务描述的忠实转述；后半部分明确给出实现建议、系统架构、里程碑和避坑点。
项目本质	不是做一个固定 demo，而是做一个能够读取评测文件、动态选择工具、自动 profiling 与解释的多智能体基础设施系统。
建议用途	将本 PDF 作为 Codex / 其他代码生成模型的系统设计输入，先搭主链路，再逐步扩展。

阅读方式 第 1 部分是题目描述与官方要求；第 2 部分是对补充要求的含义拆解；第 3 部分是建议的实现方向与编码顺序。

1. 题目描述（按官方材料整理）

项目总背景：随着 Agent Era 到来，系统工程正在从“人工手写高性能代码”转向“设计能够自主写、评估、改进代码的系统”。课程项目希望学生搭建一个面向 AI Infra 的自动化系统，覆盖 GPU profiling、CUDA kernel auto-tuning 以及自动化的 LLM infra generation。其中 Phase 1 的重点是 GPU Performance Analysis。

Phase 1 标题	GPU Performance Analysis Guide: Identifying Bottlenecks with ncu
截止时间	2026 年 4 月 21 日上午 8 点

核心工具	Nsight Compute (ncu)；补充材料还给出了 Nsight Systems (nsys)、Torch Profiler、Executor 与 LLM/Router/Server 这一整套工作流。
目标	让 Agent 读取 profiling 任务，执行 profiling，分析硬件指标，把执行结果翻译成对 GPU 架构与性能瓶颈的理解。

1.1 Phase 1 的基础任务

- 使用 ncu 对给定算子进行 kernel-level profiling，并识别性能瓶颈。
- 先用 Roofline 式的思路判断算子是 compute-bound 还是 memory-bound。
- 如果是 memory-bound，继续查看 L1 / L2 / DRAM 等存储层级指标。
- 如果是 compute-bound，继续查看 Tensor Core / FP32/FMA 等计算单元指标。
- 同时检查 occupancy、warp divergence、bank conflict 等异常。
- 最后把指标映射回 CUDA kernel / operator 的实际代码行为，例如数据复用不足、共享内存缺失、线程访问模式不佳、循环展开不足等。

1.2 官方明确提到的分析维度

维度	官方关注点
总览	sm__throughput 与 gpu__compute_memory_throughput，用于先做 compute / memory 粗分类。
存储层级	L1 cache、L2 cache、DRAM/VRAM 的利用情况，以及是否存在数据复用不足。
计算单元	Tensor Core 是否被有效触发；传统 FP32/FMA 是否成为主要执行路径。
占用与调度	理论 occupancy 与 achieved occupancy 的差距是否过大。
常见瓶颈	VRAM bound、uncoalesced access、warp divergence、bank conflict 等。

1.3 Phase 1 的高级任务：硬件内禀特性探测

在高级阶段，Agent 不再只是“读已有 kernel 的 ncu 报告”，而是要充当 **Hardware Probe**，通过自动生成并执行 micro-benchmark 去反推 GPU 的物理特性与架构极限。

- 测量 L1 / L2 / DRAM latency hierarchy。
- 测量 shared memory 与 global memory 的 effective peak bandwidth。
- 通过 latency-vs-size 曲线找到 L2 cache capacity。
- 在 sustained compute load 下测 actual boost clock。
- 量化 shared memory bank conflict 相对无冲突访问的 penalty。

1.4 输入输出与评分方式

输入	评测系统会提供 target_spec.json，至少包含 targets；补充通知进一步说明文件中还应带有 run 字段，即被评测的 executable path。
输出	Agent 需要生成 results.json，键名与目标保持对应，并给出识别得到的数值；同时最好保留完整 reasoning / logs。
评分关注	数值一致性 + 工程推理与方法论。也就是说，不仅看答案是否接近 ground truth，也看系统是不是以合理的 probing / profiling 方法得到结论。
评测机制	会把 Agent 输出、ground truth 与实验性证据一起交给高能力 LLM 进行综合评分。

1.5 官方明确提示的反作弊与环境变化

- GPU 频率可能被锁在非标准值，因此不能查网上规格表直接填答案。
- 系统可能限制可用 SM 数量或每个 block 的资源。
- 标准 API（如 cudaGetDeviceProperties）可能被拦截或返回误导性信息。
- 因此 Agent 应当采用 multi-strategy fusion：micro-benchmark + binary execution + ncu metric analysis + 交叉验证。

2. 补充要求的真实含义（结合老师额外通知）

关键判断 这个 PJ 的真实验收对象不是一个固定算子的 profiling demo，而是一个能够读评测文件、动态路由工具、自动执行 profiling / probing、并输出可解释结果的通用 agent framework。

2.1 “build a multi-agent framework” 到底意味着什么

- 系统应该具备任务理解、规划、执行、分析、反馈这几层能力，而不是只有一个大 prompt。
- 补充课件给出的 workflow 已经很明确：Task Understanding -> Profiling Plan Generation -> Profiling Execution -> Result Interpretation -> Iterative Refinement。
- 也就是说，作业期待的是一个小型 AI infra system，而不是简单命令封装。

2.2 “profiling target should not be hardcoded” 的工程含义

- Agent 不能把 operator、targets、metrics、输出字段写死在逻辑里。
- 系统必须先读取 target_spec.json，再根据其中的 targets 动态组装 profiling / probing 计划。
- 后续项目里 profiling target 会变，因此现在就要把 Phase 1 设计成 spec-driven，而不是 demo-driven。

2.3 “run: executable path” 这一补充为什么重要

- 这说明评测不是只让你“分析一个概念上的算子”，而是会给一个真实可执行对象让你去跑。
- 因此 Agent 至少需要：读取 run 字段、执行该对象、在其外层挂 ncu / nsys / 自定义 probe，并收集输出。
- 如果缺少 run 支持，你的系统就没有完整的任务入口。

2.4 这次 Phase 1 应该达成的最低闭环

闭环环节	最低要求
读任务	读懂 target_spec.json，识别 targets 与 run。
做计划	判断应该走 operator profiling 路径，还是 hardware probing 路径。
执行	调用 executable，并以 ncu / nsys / micro-benchmark 的方式采数据。
分析	把原始 trace / metric 转成 bottleneck diagnosis 或 hardware parameter estimation。
产出	输出 results.json，外加 reasoning / logs，能够支撑 LLM judge 给方法论分。

3. 思考后的指导方向（给 Codex 的实现导向）

下面这部分不是官方原文，而是基于题目要求与补充通知做出的**实现建议**。目标是帮助你先搭起一条最稳的主链路，再逐步扩展。

3.1 建议的系统分层

模块	职责	必须做到什么
Spec Reader	解析输入文件	读取 targets、run、可选参数，形成统一内部任务表示。
Planner / Router	动态组装策略	根据 targets 决定使用 ncu、nsys、Torch Profiler、还是自定义 micro-benchmark。
Executor	真实执行命令	运行 executable、调用 profiling 工具、编译并执行 probe、保存原始日志。
Analyzer	从数据到结论	做 compute / memory 归因、硬件参数估计、异常环境识别。
Reporter	稳定产出	生成 results.json 与 reasoning/logs，确保键名和 spec 对齐。

3.2 不要把它做成“一个大模型 + 一段 prompt”

- 推荐用“规则路由 + LLM 解释”而不是“纯 LLM 决定一切”。
- 工具选择、文件解析、结果落盘这些部分高度结构化，最好用确定性代码实现。
- LLM 更适合做两件事：一是把 profiling 指标转成可读解释；二是辅助生成 micro-benchmark 模板或异常诊断说明。

3.3 推荐的任务分类方式

Operator Profiling 类	例如 bottleneck diagnosis、tensor core utilization、memory hierarchy analysis。这类任务主路径应是 run -> ncu/nsys -> metric parser -> bottleneck analyzer。
-----------------------------	--

Hardware Probe 类	例如 dram_latency_cycles、l2_cache_capacity、actual_boost_clock_mhz、bank_conflict_penalty。这类任务主路径应是自动生成 probe -> 编译执行 -> 数值拟合 / 曲线分析 -> 结果回填。
Hybrid 类	某些任务既要看 ncu，也要做补充探针验证；在环境可能被锁频、masking、API 干扰时，Hybrid 路线更稳。

3.4 建议先实现的最小可交付版本 (MVP)

- 1. 先把 target_spec.json 解析器写好，确保系统不依赖写死的 target。
- 2. 实现 Executor，能够正确读取 run 并执行目标程序。
- 3. 先打通 ncu 路线：执行目标 -> 抽取关键 metrics -> 粗分类 compute-bound / memory-bound。
- 4. 在此基础上加入二级分析：memory hierarchy、tensor core、occupancy、bank conflict。
- 5. 定义统一输出格式 results.json，并额外保存 reasoning.txt 或 logs/。
- 6. 再逐步加入 micro-benchmark 生成能力，用于 latency、bandwidth、clock、cache size 等更高级 target。
- 7. 最后加入 cross-verification：重复试验、多类 probe 比对、检测非标准频率与资源 masking。

3.5 一个更稳的 repo 组织方式

```
project/  
  main.py  
  schemas/ target_spec.schema.json, results.schema.json  
  agents/ spec_reader.py, planner.py, analyzer.py, verifier.py, reporter.py  
  executors/ run_target.py, run_ncu.py, run_nsys.py, compile_probe.py, run_probe.py  
  probes/ pointer_chasing.cu, bandwidth_sweep.cu, bank_conflict.cu, clock_probe.cu  
  parsers/ parse_ncu.py, parse_nsys.py, parse_stdout.py  
  outputs/ results.json, reasoning.txt, traces/
```

3.6 需要特别避免的误区

误区	为什么危险
把 matmul 写死	评测 target 会变化，系统会立刻失去泛化能力。

只会解释 ncu，不会执行目标	你可能连 run 指向的程序都没真正跑起来。
只依赖 API 或网上规格表	锁频、SM masking、虚拟化返回值会导致结果失真甚至直接归零。
只输出数字，不保存证据链	LLM judge 会在 methodology 部分扣分，难以给高分。
把所有逻辑塞进 prompt	可复现性差，调试困难，也不利于 Phase 2/3 扩展。

3.7 给 Codex 的直接 coding 任务描述

可以直接投喂给代码模型 请实现一个 spec-driven 的 GPU profiling multi-agent framework。输入为 target_spec.json，至少包含 targets 与 run 字段；系统需要读取任务、动态决定 profiling 路径、执行 executable、调用 ncu / nsys / 自定义 probe、解析结果并输出 results.json。要求不要对 operator、target、metrics 或 executable path 进行硬编码；同时保留 reasoning/logs，以便后续 LLM 评分使用。优先实现一条最小闭环：spec reader -> planner -> executor -> ncu parser -> analyzer -> reporter，再逐步加入 micro-benchmark、环境异常检测与 cross-verification。

4. 建议的开发里程碑

阶段	目标	完成标志
Milestone 1	能读 spec 并执行 run	给定 target_spec.json 后，系统能够正确启动目标程序并保存日志。
Milestone 2	能跑 ncu 并做基础归因	至少可以识别 compute-bound / memory-bound，并输出解释。
Milestone 3	能适配多 target	不同 targets 会触发不同 profiling / probing 路径，而不是写死同一套命令。

Milestone 4	能做 micro-benchmark	支持 latency / bandwidth / bank conflict / clock 这类更硬件化的目标。
Milestone 5	能抵抗评测扰动	通过重复试验与交叉验证识别锁频、SM masking、API 误导等环境差异。

5. 最后的判断

这次 Phase 1 最核心的不是“你会不会看一张 ncu 图”，而是：

- 你能不能把 profiling 问题抽象成一个可以自动读任务、自动路由工具、自动执行、自动解释、自动输出结果的系统。
- 只要把这条主链搭起来，后面的 Phase 2 和 Phase 3 才真正有可扩展的基础。
- 如果只做一个固定 demo，短期看起来快，后面大概率需要整体返工。

一句话总结 先把 spec-driven 的主链做对，再逐步增加更复杂的 probe 和推理能力。